StudentPay AI — Technical Documentation

Competition submission document • current prototype architecture, algorithms, security design, limitations and verification

1. Project Overview

StudentPay AI is a browser-based educational finance prototype for students, especially students handling cross-border transfers and everyday budgeting. The submitted implementation is a self-contained HTML/CSS/JavaScript prototype. It does not execute real payments and explicitly presents itself as an educational prototype; rates and fees are manually entered demo values and data is stored only in the user's browser.

Important scope boundary: This document describes the implementation that actually exists in the submitted HTML. It does not claim a production backend, real payment rails, real-time exchange-rate feeds, regulated financial advice, or production-grade authentication.

2. Functional Modules

Module	Purpose	Implementation
Authentication	Create and log into a local prototype account.	Client-side JavaScript; credentials stored in localStorage as salted SHA-256 hashes.
Transfer comparison	Compare manually entered providers by rate, fee and estimated recipient amount.	Deterministic arithmetic in calc() and compare().
Budget / affordability	Calculate expenses, remaining funds and a simple budget warning.	Client-side calculation; saved per user in localStorage.
Survival planner	Forecast whether essentials can be covered until expected income.	balance + income − bills − daily essentials × days.
Transactions	Record income and expenses and export CSV.	localStorage-backed list; CSV generated in browser.
Bills	Track upcoming, due-today and overdue bills.	localStorage-backed list with paid/unpaid state.
Savings goals	Track target, current savings and monthly amount needed.	Client-side arithmetic.
Dashboard	Summarize budget, goals, comparisons, transactions and bills.	Derived client-side score and tips.
AI assistant	Answer questions using entered prototype data.	Attempts a provided Claude sample connection; falls back to a deterministic local advisor.
Scam shield	Flag common warning signs before a transfer.	Checkbox-based risk heuristic; not a real provider verification service.
Activity log	Record selected prototype events.	localStorage audit array, displayed for the current user.

3. System Architecture

The architecture is intentionally lightweight. There is one browser application containing the presentation layer, application logic and persistence layer. There is no custom server or database in the submitted source.

- Presentation layer: HTML structure, CSS variables, cards, tabs, forms, tables and status messages.
- Application layer: JavaScript functions for authentication, calculations, state transitions, rendering, validation, CSV export and the AI-assistant integration.
- Persistence layer: browser localStorage under the key sp_v1. The DB object contains users, saved comparisons, audit entries, login attempts and optional module data such as budgets, goals, transactions, bills and survival estimates.
- External/runtime integration: the AI assistant attempts to use window.claude.use("sample"). If unavailable, the application falls back to localAI(), so the prototype remains usable without that connection.
- No payment processor, bank API, exchange-rate API, server database or custom backend is present in the submitted source.

See the separate System_Architecture.svg and Data_and_User_Flow.svg files in the diagrams folder.

4. Data Model

Object	Key fields	Purpose
DB.users[user]	name, email, salt, hash	Prototype account identity and password verifier.
DB.attempts[user]	timestamps[]	Tracks failed login attempts for a temporary local lockout.
DB.saved[user]	id, title, at, data	Saved transfer comparisons.
DB.budget[user]	inc, tot, rem, ex	Saved affordability calculation.
DB.goals[user]	id, name, target, saved, months	Savings goal records.
DB.transactions[user]	id, desc, type, category, amount, date, currency	Income/expense records.
DB.bills[user]	id, name, amount, due, category, currency, paid	Bill and tuition records.
DB.survival[user]	balance, days, income, daily, bills, cur, at	Latest survival forecast inputs.
DB.audit[]	t, u, ev, detail	Selected activity events.

5. Algorithm Descriptions

The detailed step-by-step algorithms are provided in the separate Algorithm_Description.docx. The core transfer algorithm is:

1. Read transfer amount, sending currency, recipient currency and manually entered mid-market rate.
2. Read each provider's exchange rate, fee amount and whether the fee is charged in the sending or recipient currency.

3. Validate amount > 0, rate > 0, fee ≥ 0 and fee smaller than the relevant amount.
4. If the fee is in sending currency, calculate received = (amount − fee) × rate.
5. If the fee is in recipient currency, calculate received = amount × rate − fee.
6. Sort providers from highest estimated recipient amount to lowest.
7. Use the top provider as the current best option and calculate the difference versus the best.
8. When a positive mid-market rate is supplied, estimate hidden cost as ideal recipient amount − provider recipient amount.
9. Render the result table and allow the comparison to be saved.

6. Validation and Error Handling

- Registration validates username format, email format and minimum password requirements.
- Login limits failed attempts to five within a rolling 15-minute window for the local prototype.
- Financial inputs reject invalid numbers, negative values and zero/invalid transfer amounts where appropriate.
- HTML output that originates from user-controlled text is escaped through esc() before being inserted into innerHTML.
- Module-level messages communicate errors, warnings and successful actions to the user.
- The prototype deliberately labels financial outputs as estimates and not financial advice.

7. Security Design

The current security design is appropriate only for a local educational prototype. Passwords are not stored in plaintext: a random salt is generated and a SHA-256 digest of salt + password is stored. However, because all code and localStorage data are client-side, this is not equivalent to production authentication. A malicious script running in the same origin could access localStorage and application state.

- Prototype password handling: random per-user salt plus SHA-256 digest.
- Input validation: username, email, password and numeric finance fields are validated.
- Output encoding: esc() reduces HTML injection through displayed user-entered values.
- Login throttling: five failed attempts within 15 minutes causes a local temporary block.
- Data minimization: the prototype uses locally entered demo data and does not implement bank credentials or payment credentials.
- Privacy boundary: localStorage is browser-local, but it should not be described as secure storage for sensitive financial data.
- No server-side authorization exists; production deployment would require a backend, HTTPS, secure cookies/session tokens, server-side validation and access control.
- The scam shield is advisory and heuristic; it does not verify licenses or providers against an authoritative registry.

8. Known Limitations / Risks

Area	Current state	Required for production
Backend	None	API/server, database, authentication service and authorization.
Authentication	Local prototype hashing	Argon2id/bcrypt/scrypt server-side password hashing, secure sessions, MFA as appropriate.
Storage	localStorage	Encrypted/controlled server-side storage with access controls.
Payments	No real payments	Licensed payment/remittance provider integration and compliance.
Exchange rates	Manual demo input	Trusted live rate provider with timestamp/source and failure handling.
AI	Optional runtime integration + local fallback	Managed API, data-minimization policy, prompt-injection controls, monitoring and provider terms review.
Security verification	Static self-assessment	Penetration test, dependency scanning, threat modeling and security review.
Notifications	In-page reminders only	Permissioned notification service if required.
Financial advice	Educational estimates	Regulatory/legal review if recommendations become financial advice.

9. Technical Verification Performed

- The submitted HTML was parsed and its JavaScript block was checked with Node.js syntax validation; no syntax error was reported.
- Static reference validation found no missing document.getElementById-style IDs among the IDs referenced by the submitted JavaScript.
- The project contains the required prototype functions for registration, login, transfer comparison, budget, survival planning, transactions, bills, goals, dashboard, AI assistant, scam shield and activity log.
- Manual UI execution evidence is still required for the competition submission and should be captured from the actual running prototype.

Evidence rule: Do not claim that a test passed from this document alone unless you actually run the prototype and record the observed result. The separate test plan marks which cases require live screenshots/logs.

10. Reproducibility

10. Open StudentPay_AI_Enhanced.html in a modern browser.
11. Create a prototype account or use an account created in that browser.

12. Use the tabs to exercise each module.
13. For a clean run, clear site data/localStorage for the file/origin and reload.
14. The prototype is self-contained except for the optional AI runtime connection represented by window.claude.use("sample").